

Piecewise Position Optimization with CVXPY

May 22, 2025

1 Problem Definition

We optimize a position $W(t)$ over time horizon T with piecewise costs:

$$f(t) = \begin{cases} \Delta W \times (\text{ask} - B) & \text{Case A1: } \Delta W < 0, W(t) < 0 \\ -\max(-W(t+1)) \times (\text{ask} - B) - \Psi \times (B - \text{bid}) & \text{Case A2: } \Delta W < 0, W(t) > 0 \\ \Delta W \times (\text{bid} - B) & \text{Case B1: } \Delta W > 0, W(t) > 0 \\ \max(W(t+1)) \times (\text{bid} - B) + \Psi \times (B - \text{ask}) & \text{Case B2: } \Delta W > 0, W(t) < 0 \end{cases}$$

where $\Psi = W(t) + \min(0, W(t+1))$.

2 Solution Architecture

2.1 Core Components

- **Position Decomposition:**

$$\begin{aligned} W(t) &= W^+(t) - W^-(t) \\ W^+(t) &= \max(W(t), 0) \\ W^-(t) &= \max(-W(t), 0) \end{aligned}$$

- **Regime Selection:** Binary variables enforce mutual exclusivity:

$$\sum_{i \in \{A1, A2, B1, B2\}} z_i(t) = 1 \quad \forall t$$

2.2 Mathematical Reformulation

Each regime's cost becomes:

$$\text{Case A1: } c_{A1}(t) = \Delta W(t) \times (\text{ask} - B)$$

$$\text{Case A2: } c_{A2}(t) = -W^-(t+1) \times (\text{ask} - B) - (W(t) - W^-(t+1)) \times (B - \text{bid})$$

$$\text{Case B1: } c_{B1}(t) = \Delta W(t) \times (\text{bid} - B)$$

$$\text{Case B2: } c_{B2}(t) = W^+(t+1) \times (\text{bid} - B) + (W(t) - W^-(t+1)) \times (B - \text{ask})$$

3 Implementation Details

3.1 Initialization

```
1 # User inputs (constants)
2 T = 10          # Time horizon
3 ask = 1.2        # Ask price
4 bid = 1.0        # Bid price
5 B = 1.1         # Benchmark price
6 M = 1e5         # Big-M constant
7 eps = 1e-5      # Threshold for strict inequalities
8
9 # Precompute price differences
10 cost_ask = ask - B # Constant
11 cost_bid = bid - B # Constant
```

3.2 Variable Definition

```
1 # Core variables
2 W = cp.Variable(T)          # Position at time t
3 dW = W[1:] - W[:-1]        # Position change W
4
5 # Decomposition variables
6 W_plus = cp.Variable(T)    # W(t) = max(W(t), 0)
7 W_minus = cp.Variable(T)   # W(t) = max(-W(t), 0)
8
9 # Binary regime selectors
10 z_A1 = cp.Variable(T-1, boolean=True)
11 z_A2 = cp.Variable(T-1, boolean=True)
12 z_B1 = cp.Variable(T-1, boolean=True)
13 z_B2 = cp.Variable(T-1, boolean=True)
14
15 # Auxiliary cost variables
16 cost_A1 = cp.Variable(T-1)
17 cost_A2 = cp.Variable(T-1)
18 cost_B1 = cp.Variable(T-1)
19 cost_B2 = cp.Variable(T-1)
```

3.3 Constraint System

```
1 constraints = [
2     # Position decomposition
3     W == W_plus - W_minus,
4     W_plus >= 0,
5     W_minus >= 0,
6
7     # Exactly one regime active per timestep
8     [z_A1[t] + z_A2[t] + z_B1[t] + z_B2[t] == 1 for t in range(T-1)],
9
10    # Case A1 constraints
11    [cost_A1[t] == dW[t] * cost_ask for t in range(T-1)],
12    [dW[t] <= -eps + M*(1 - z_A1[t]) for t in range(T-1)],
```

```

13 [W[t] <= -eps + M*(1 - z_A1[t]) for t in range(T-1)],
14 [cost_A1[t] <= M * z_A1[t] for t in range(T-1)],
15
16 # Case A2 constraints
17 [cost_A2[t] == (-W_minus[t+1])*cost_ask - (W[t]-W_minus[t+1])
18 *(-cost_bid) for t in range(T-1)],
19 [dW[t] <= -eps + M*(1 - z_A2[t]) for t in range(T-1)],
20 [W[t] >= eps - M*(1 - z_A2[t]) for t in range(T-1)],
21 [cost_A2[t] <= M * z_A2[t] for t in range(T-1)],
22
23 # ... (similar for B1/B2 cases)
24 ]

```

3.4 Objective and Solving

```

1 total_cost = cp.sum(cost_A1 + cost_A2 + cost_B1 + cost_B2)
2 problem = cp.Problem(cp.Minimize(total_cost), constraints)
3 problem.solve(solver=cp.MOSEK, verbose=True)

```

4 Key Techniques

4.1 Big-M Method

For each regime i at time t :

If $z_i(t) = 0 \Rightarrow$ Constraints become $|c_i(t)| \leq 0$

If $z_i(t) = 1 \Rightarrow$ Original constraint active

4.2 Position Decomposition

Algorithm 1 Handle max/min operations

- 1: $W^+(t) \geq W(t)$
 - 2: $W^+(t) \geq 0$
 - 3: $W^-(t) \geq -W(t)$
 - 4: $W^-(t) \geq 0$
 - 5: Exact through minimization in objective
-

5 Results Interpretation

Output includes:

- Optimal positions $W^*(t)$
- Active regime sequence
- Cost breakdown by regime

$$\text{Total Cost} = \sum_{t=0}^{T-1} \sum_{i \in \mathcal{R}} z_i(t) c_i(t) \quad (1)$$

where $\mathcal{R} = \{A1, A2, B1, B2\}$.

6 Problem Definition

We optimize a position $W(t)$ over time horizon T with piecewise costs and no-move option:

$$f(t) = \begin{cases} 0 & \text{Case N: No move } (\Delta W = 0) \\ \Delta W \times (\text{ask} - B) & \text{Case A1: } \Delta W < 0, W(t) < 0 \\ -\max(-W(t+1)) \times (\text{ask} - B) - \Psi \times (B - \text{bid}) & \text{Case A2: } \Delta W < 0, W(t) > 0 \\ \Delta W \times (\text{bid} - B) & \text{Case B1: } \Delta W > 0, W(t) > 0 \\ \max(W(t+1)) \times (\text{bid} - B) + \Psi \times (B - \text{ask}) & \text{Case B2: } \Delta W > 0, W(t) < 0 \end{cases}$$

where $\Psi = W(t) + \min(0, W(t+1))$.

7 Solution Architecture

7.1 Core Components

- **Position Decomposition:**

$$\begin{aligned} W(t) &= W^+(t) - W^-(t) \\ W^+(t) &= \max(W(t), 0) \\ W^-(t) &= \max(-W(t), 0) \end{aligned}$$

- **Regime Selection:** Binary variables enforce mutual exclusivity:

$$z_N(t) + \sum_{i \in \{A1, A2, B1, B2\}} z_i(t) = 1 \quad \forall t$$

- **No-Move Enforcement:**

$$|\Delta W(t)| \leq M(1 - z_N(t))$$

8 Complete Implementation

8.1 Initialization and Parameters

```

1 import cvxpy as cp
2 import numpy as np
3
4 # Parameters
5 T = 10                # Time horizon
6 ask = 1.2              # Ask price
7 bid = 1.0              # Bid price
8 B = 1.1                # Benchmark price
9 M = 1e5                # Big-M constant
10 eps = 1e-5            # Threshold for strict inequalities
11 inactivity_penalty = 0.001 # Penalty for no-move actions
12
13 # Precompute price differences
14 cost_ask = ask - B      # Constant
15 cost_bid = bid - B      # Constant

```

8.2 Variable Definition

```

1 # Core variables
2 W = cp.Variable(T)      # Position at time t
3 dW = W[1:] - W[:-1]    # Position change W
4
5 # Decomposition variables
6 W_plus = cp.Variable(T) # W(t) = max(W(t), 0)
7 W_minus = cp.Variable(T) # W(t) = max(-W(t), 0)
8
9 # Regime selection variables
10 z_N = cp.Variable(T-1, boolean=True) # No-move regime
11 z_A1 = cp.Variable(T-1, boolean=True) # Case A1
12 z_A2 = cp.Variable(T-1, boolean=True) # Case A2
13 z_B1 = cp.Variable(T-1, boolean=True) # Case B1
14 z_B2 = cp.Variable(T-1, boolean=True) # Case B2
15
16 # Cost variables
17 cost_N = cp.Variable(T-1) # No-move cost (0)
18 cost_A1 = cp.Variable(T-1) # Case A1 cost
19 cost_A2 = cp.Variable(T-1) # Case A2 cost
20 cost_B1 = cp.Variable(T-1) # Case B1 cost
21 cost_B2 = cp.Variable(T-1) # Case B2 cost

```

8.3 Constraint System

```

1 constraints = [
2     # Position decomposition
3     W == W_plus - W_minus,
4     W_plus >= 0,
5     W_minus >= 0,
6
7     # Exactly one regime active per timestep
8     [z_N[t] + z_A1[t] + z_A2[t] + z_B1[t] + z_B2[t] == 1
9     for t in range(T-1)],
10
11     # No-move constraints
12     [dW[t] <= M * (1 - z_N[t]) for t in range(T-1)],

```

```

13 [dW[t] >= -M * (1 - z_N[t]) for t in range(T-1)],
14 [cost_N[t] == 0 for t in range(T-1)],
15
16 # Case A1: W < 0 and W(t) < 0
17 [cost_A1[t] == dW[t] * cost_ask for t in range(T-1)],
18 [dW[t] <= -eps + M*(1 - z_A1[t]) for t in range(T-1)],
19 [W[t] <= -eps + M*(1 - z_A1[t]) for t in range(T-1)],
20 [cost_A1[t] <= M * z_A1[t] for t in range(T-1)],
21 [cost_A1[t] >= -M * z_A1[t] for t in range(T-1)],
22
23 # Case A2: W < 0 and W(t) > 0
24 [cost_A2[t] == (-W_minus[t+1])*cost_ask -
25 (W[t]-W_minus[t+1])*(-cost_bid)
26 for t in range(T-1)],
27 [dW[t] <= -eps + M*(1 - z_A2[t]) for t in range(T-1)],
28 [W[t] >= eps - M*(1 - z_A2[t]) for t in range(T-1)],
29 [cost_A2[t] <= M * z_A2[t] for t in range(T-1)],
30 [cost_A2[t] >= -M * z_A2[t] for t in range(T-1)],
31
32 # Case B1: W > 0 and W(t) > 0
33 [cost_B1[t] == dW[t] * cost_bid for t in range(T-1)],
34 [dW[t] >= eps - M*(1 - z_B1[t]) for t in range(T-1)],
35 [W[t] >= eps - M*(1 - z_B1[t]) for t in range(T-1)],
36 [cost_B1[t] <= M * z_B1[t] for t in range(T-1)],
37 [cost_B1[t] >= -M * z_B1[t] for t in range(T-1)],
38
39 # Case B2: W > 0 and W(t) < 0
40 [cost_B2[t] == W_plus[t+1]*cost_bid +
41 (W[t]-W_minus[t+1])*(-cost_ask)
42 for t in range(T-1)],
43 [dW[t] >= eps - M*(1 - z_B2[t]) for t in range(T-1)],
44 [W[t] <= -eps + M*(1 - z_B2[t]) for t in range(T-1)],
45 [cost_B2[t] <= M * z_B2[t] for t in range(T-1)],
46 [cost_B2[t] >= -M * z_B2[t] for t in range(T-1)]
47 ]

```

8.4 Objective and Solving

```

1 # Objective with inactivity penalty
2 total_cost = cp.sum(
3     cost_N + cost_A1 + cost_A2 + cost_B1 + cost_B2 +
4     inactivity_penalty * cp.sum(z_N) # Penalize no-moves
5 )
6
7 problem = cp.Problem(cp.Minimize(total_cost), constraints)
8 problem.solve(solver=cp.MOSEK, verbose=True)
9
10 # Results analysis
11 active_regime = np.argmax(
12     np.vstack([
13         z_N.value,
14         z_A1.value,
15         z_A2.value,
16         z_B1.value,
17         z_B2.value
18     ]).T,

```

```

19     axis=1
20 )
21 regime_names = [
22     "No move",
23     "A1: W <0, W<0",
24     "A2: W <0, W>0",
25     "B1: W >0, W>0",
26     "B2: W >0, W<0"
27 ]
28
29 print("Optimal positions:", W.value)
30 print("\nRegime sequence:")
31 for t, r in enumerate(active_regime):
32     print(f"t={t}: {regime_names[r]}")

```

9 Key Techniques

9.1 Big-M Method

For each regime $i \in \{N, A1, A2, B1, B2\}$:

If $z_i(t) = 0 \Rightarrow$ Constraints become inactive

If $z_i(t) = 1 \Rightarrow$ Regime constraints enforced

9.2 Inactivity Penalty

- Added term $\alpha \sum_t z_N(t)$ to objective
- α controls no-move frequency (set to 0.001)
- Ensures no-moves only when beneficial

10 Results Interpretation

Output includes:

- Optimal positions $W^*(t)$
- Regime sequence with no-move identification
- Cost breakdown by regime type
- Visualization of position changes vs. no-move periods

$$\text{Total Cost} = \sum_{t=0}^{T-1} \left(\sum_{i \in \mathcal{R}} z_i(t) c_i(t) \right) + \alpha \sum_t z_N(t) \quad (2)$$

where $\mathcal{R} = \{A1, A2, B1, B2\}$.

11 Problem Statement

We optimize a position trajectory $W(t)$ over a time horizon T while minimizing piecewise-defined transaction costs based on whether the position increases or decreases and whether it's crossing the zero boundary (changing trade direction).

The cost function is defined as follows:

$$f(t) = \begin{cases} 0 & \text{(N) No move: } \Delta W = 0 \\ \text{(A1) Sell in short: } \Delta W < 0, W(t) < 0 & -\max(-W(t+1))(\text{ask} - B) - \Psi(B - \text{bid}) & \text{(A2) Sell from long: } \Delta W < 0, W(t) > 0 \\ \text{(B1) Buy in long: } \Delta W > 0, W(t) > 0 & \max(W(t+1))(\text{bid} - B) + \Psi(B - \text{ask}) & \text{(B2) Buy from short: } \Delta W > 0, W(t) < 0 \end{cases}$$

where $\Psi = W(t) + \min(0, W(t+1))$.

12 Solution Approach

To encode the piecewise cost, we:

- Define five regimes (N, A1, A2, B1, B2), using binary variables $z_i(t)$ to activate each.
- Use Big-M constraints to enforce conditional logic per regime.
- Use decomposition variables $W^+(t), W^-(t)$ for absolute value modeling.
- Write cost terms using affine expressions, which allow signed inputs (e.g., $\text{ask} - B$).

13 Python Code

```

1 import cvxpy as cp
2 import numpy as np
3
4 T = 10
5 ask = 1.2
6 bid = 1.0
7 B = 1.1
8 M = 1e5
9 eps = 1e-5
10 penalty = 0.001
11
12 cost_ask = ask - B
13 cost_bid = bid - B
14
15 W = cp.Variable(T)
16 dW = W[1:] - W[:-1]
17 W_plus = cp.Variable(T)
18 W_minus = cp.Variable(T)
19
20 z_N = cp.Variable(T-1, boolean=True)

```



```

21 z_A1 = cp.Variable(T-1, boolean=True)
22 z_A2 = cp.Variable(T-1, boolean=True)
23 z_B1 = cp.Variable(T-1, boolean=True)
24 z_B2 = cp.Variable(T-1, boolean=True)
25
26 cost_N = cp.Variable(T-1)
27 cost_A1 = cp.Variable(T-1)
28 cost_A2 = cp.Variable(T-1)
29 cost_B1 = cp.Variable(T-1)
30 cost_B2 = cp.Variable(T-1)
31
32 constraints = [
33 W == W_plus - W_minus,
34 W_plus >= 0,
35 W_minus >= 0,
36 [z_N[t] + z_A1[t] + z_A2[t] + z_B1[t] + z_B2[t] == 1 for t in range
    (T-1)],
37 [dW[t] <= M * (1 - z_N[t]) for t in range(T-1)],
38 [dW[t] >= -M * (1 - z_N[t]) for t in range(T-1)],
39 [cost_N[t] == 0 for t in range(T-1)],
40 [cost_A1[t] == dW[t] * cost_ask for t in range(T-1)],
41 [dW[t] <= -eps + M * (1 - z_A1[t]) for t in range(T-1)],
42 [W[t] <= -eps + M * (1 - z_A1[t]) for t in range(T-1)],
43 [cost_A2[t] == (-W_minus[t+1]) * cost_ask - (W[t] - W_minus[t+1]) *
    (-cost_bid) for t in range(T-1)],
44 [dW[t] <= -eps + M * (1 - z_A2[t]) for t in range(T-1)],
45 [W[t] >= eps - M * (1 - z_A2[t]) for t in range(T-1)],
46 [cost_B1[t] == dW[t] * cost_bid for t in range(T-1)],
47 [dW[t] >= eps - M * (1 - z_B1[t]) for t in range(T-1)],
48 [W[t] >= eps - M * (1 - z_B1[t]) for t in range(T-1)],
49 [cost_B2[t] == W_plus[t+1] * cost_bid + (W[t] - W_minus[t+1]) * (-
    cost_ask) for t in range(T-1)],
50 [dW[t] >= eps - M * (1 - z_B2[t]) for t in range(T-1)],
51 [W[t] <= -eps + M * (1 - z_B2[t]) for t in range(T-1)]
52 ]
53
54 total_cost = cp.sum(cost_N + cost_A1 + cost_A2 + cost_B1 + cost_B2
    + penalty * cp.sum(z_N))
55 problem = cp.Problem(cp.Minimize(total_cost), constraints)
56 problem.solve(solver=cp.MOSEK)
57
58 active_regime = np.argmax(np.vstack([z_N.value, z_A1.value, z_A2.
    value, z_B1.value, z_B2.value]).T, axis=1)
59 regime_names = ["No move", "A1", "A2", "B1", "B2"]
60
61 print("Optimal W:", W.value)
62 for t, r in enumerate(active_regime):
63     print(f"t={t}: {regime_names[r]}")

```

14 Discussion

- **Generalized to Sign-agnostic Inputs:** The model allows any user-defined prices (ask, bid, B), even if they produce negative spreads.
- **Optimization-safe Logic:** All piecewise branches are activated using

DCP-compliant expressions.

- **Extensible:** New regimes or soft constraints can be added easily.